

Процесс разработки продукта: упрощённый вариант

Это предложение, как из большого списка на доске Miro — 7 шагов и 89 пунктов — сделать процесс, который реально можно соблюдать. Внутри: пять этапов, в каждом — вход, кто что делает, выход и правила; раскладка по двухнедельному спринту; разбор куда переехал каждый пункт из Miro; живой пример на одной фиче целиком; в приложении — примеры заполненных шаблонов.

Разбор опирается на три вещи: принципы, которые видны у больших компаний (один обязательный артефакт на входе этапа, решения на входе, жёсткий выход из цикла, обязательность живёт в инструменте, а не в тексте), замеры по нашей доске и разбор пунктов по типам.

1. Что меняется, если коротко

Было в Miro	Стало
7 шагов, часть из них перекрывается	5 этапов, каждый заканчивается одним артефактом
89 пунктов одним списком	~25 шагов работы, ~15 правил-договорённостей, остальное — отдельные планы
Цепочка согласований по шагам	Два гейта: «берём в работу» и «готово к продю»
58 строк с числовыми метриками	3 метрики процесса + 1 метрика продукта на инициативу
Дизайн растворён в шагах 3, 4 и 5	Дизайн — отдельный этап с входом, выходом и правилами
Внедрение процесса внутри процесса	Отдельный план раскатки, не мешает продукту

Главное: **это два разных плана, склеенных в один список**. «Как делаем продукт» — 25 шагов и 15 правил. «Как внедряем процесс» — отдельная история со своим графиком. Пока они вместе, любой разговор начинается с «89 пунктов — это нереально» вместо обсуждения дела.

2. Пять этапов

Этап 1. Понять, что болит

Вход: сигнал из любого канала — поддержка, Pугus, Jira, клиент, продакт.

Кто что делает:

- **Продакт** — собирает сигналы, формулирует проблему словами заказчика с источником и бизнес-контекстом, принимает решение по трём исходам.
- **Поддержка** — приносит сигнал в общую форму, не пересказывая своими словами и не обещая клиенту сроков.
- **Бизнес-аналитик** — подсказывает, где посмотреть связанные задачи и что уже было по этой теме раньше.

- **Hermes** — собирает бриф по Jira, Confluence и Pyrus: сколько обращений, от кого, что уже поднималось. Черновик, который продакт проверяет.
- **Куратор процесса** — раз в две недели видит, сколько проблем висит без решения, и поднимает это на общем разборе.

Выход: проблема с источником и бизнес-контекстом, и одно из трёх решений:

- **подтверждена** — болит у нескольких клиентов и повторяется → идём дальше;
- **не подтверждена** — единичный случай, симптом без причины → откладываем до лучших времён, не копаем;
- **есть готовое решение** — проблема уже решается в другой части системы → сразу задача, без ресёрча.

Правило выхода — самое ценное в исходной таблице (ICE 448): оно прекращает работу над тем, что никому не болит.

Этап 2. Решить, что проверяем

Вход: подтверждённая проблема.

Кто что делает:

- **Продакт** — пишет гипотезу по шаблону (проблема, метрика успеха, способ проверки, срок), решает, какие 1–3 гипотезы идут в спринт.
- **Системный аналитик** — достаёт данные из систем, если проверка идёт цифрами.
- **Тимлид и разработка** — дают оценку реализуемости, чтобы не улететь в идею, которую нельзя сделать.
- **Дизайнер** — по запросу помогает собрать черновой прототип для разговора с клиентом; черновик делает продакт, дизайнер подключается, когда нужна аккуратность или уже видно спорное место.
- **Hermes** — оформляет гипотезу по шаблону и прогоняет приоритизацию RICE; результат проверяет продакт.
- **Куратор процесса** — следит, чтобы в спринт не проходили гипотезы без плана проверки.

Выход: гипотеза по шаблону + вердикт по проверке — подтвердилась, опровергнута, что уточнили.

Внутри этапа два разных момента, и их важно не путать:

1. **Собираем и приоритизируем.** Шаблон гипотезы один для всех, критерии приоритизации согласованы командой один раз, дальше просто применяем.
2. **Проверяем.** Данные из систем, 5–7 коротких интервью или черновой прототип.

Правило-фильтр: гипотеза без плана проверки в спринт не берётся, а гипотеза, по которой к концу спринта нет вердикта, не переезжает дальше автоматически — её заново защищают или закрывают (см. раздел 3). Справочник, который экономит время: матрица «тип гипотезы → способ проверки» — боль проверяем интервью, спрос данными, понятность прототипом, поведение метрикой, реализуемость оценкой с разработкой.

Этап 3. Описать до того, как рисовать

Вход: проверенная гипотеза, по которой решение принято.

Кто что делает:

- **Продакт** — пишет Story или Epic и отвечает за обязательные поля: источник обращения, бизнес-контекст, бизнес-эффект.
- **Бизнес-аналитик** — формулирует бизнес-контекст и ожидаемый эффект, помогает с границами задачи.
- **Системный аналитик** — для комплексных историй собирает схему as is/to be в Mermaid по текстовому описанию.
- **Дизайнер** — читает Story до начала работы и говорит, чего не хватает: сценариев, состояний, критериев «где заканчивается экран». Это дешевле, чем выяснить то же самое на макете.
- **QA** — смотрит, можно ли по написанному проверить результат; если нельзя — возвращает на доработку сразу.
- **Hermes** — черновик описания по шаблону и уточняющие вопросы к нему.

Выход: Story с обязательными полями; для комплексных — схема процесса. Правила этапа: обязательные поля без исключений (это закрывает три боли разом — «непонятно, зачем делаем», «нет ссылки на источник», «нет эффекта»); порядок работы один и тот же — конкуренты → схема → прототип → описание; критерии ветвления (нужны макеты? есть прототип? нужна спека?) описаны, а не решаются на глаз.

Этап 4. Дизайн

Вход: задача на дизайн, сгенерированная из Story (второй раз то же самое не переписываем), макет текущего состояния и гайды.

Кто что делает:

- **Дизайнер** — собирает макет по гайдам: компоненты ДС, UX-паттерны (навигация, фильтры, таблицы, пустые состояния), тексты в интерфейсе. На макет — 1–2 дня. Тексты и данные от ИИ вычитывает и утверждает сам. Проходит ревью и пишет саммари для разработки: какие состояния и граничные случаи легко упустить при реализации, где поведение неочевидно из макета. Не переписывает Story в задачу и не уходит в четвёртый круг доработок.
- **Дизайнер-ревьюер** — проверяет макет по гайдам ДС и UX до передачи в разработку; цикл «ревью → доработка» ограничен тремя итерациями, дальше решение, а не четвёртый круг.
- **Продакт** — смотрит, что макет действительно решает заявленную проблему, а не просто красиво выглядит.
- **Системный аналитик** — отдаёт дизайнеру состояния и сценарии из спеки и проверяет, что ничего не потерялось.
- **QA** — проверяет, что состояния и граничные случаи, описанные в макете и саммари, можно проверить руками.
- **Hermes** — черновик текстов и данных для макета, проверка макета по гайдам как подсказка, а не как приговор.

Выход: макет + ревью по гайдам + саммари для разработки. Для крупных историй дизайн идёт на спринт раньше разработки: макет готов до груминга того спринта, в который задача попадает (см. раздел 3). Саммари — не описание фичи, а предупреждение о граблях: оно снимает вопросы на груминге.

Этап 5. Разработка, запуск, наблюдение

Вход: Story по шаблону + макет + саммари.

Кто что делает:

- **Тимлид** — проводит груминг, заводит задачи в Jira со спринтом, принимает технические решения, организует промежуточную приёмку на крупных задачах.
- **Разработчик** — делает задачу; вопросы по поведению закрывает поиском по спеке, а не сообщением дизайнеру в личку, и не додумывает поведение сам, если в спеке есть ответ.
- **Системный аналитик** — заводит задачи на реализацию и проверку каждой НФТ, поддерживает спеку в актуальном состоянии, дополняет её по итогам вопросов.
- **QA** — проверяет по критериям приёмки, ведёт приёмку, фиксирует дефекты; критерии не додумывает, а проверяет по написанному.
- **Продакт** — ведёт гейт «готово к продю»: чек-лист, решение, протокол. До запуска фиксирует метрику и карту событий, после — решение о запуске с финишем наблюдения.
- **Дизайнер** — подключается только к спорным состояниям, если при реализации нашлось упущение.
- **Поддержка** — ведёт пилот и приносит обратную связь клиентов.
- **Куратор процесса** — следит, что гейты реально проходят, а не обходятся.
- **Hermes** — отвечает по спеке и смежным спекам (ответ проверяет владелец спеки), собирает сводку результатов пилота.

Выход: работающий запуск с зафиксированной метрикой и решением по наблюдению.

- **Промежуточная приёмка** на крупных задачах — чтобы «делают не то» выяснилось в середине, а не на финале.
- **Гейт «готово к продю»:** чек-лист, решение, протокол; метрика и карта событий — до запуска, не после.
- **Пилот и наблюдение:** срок, критерии, лимит циклов и владелец решения определены заранее; есть стоп-условия для досрочной приостановки; пост-мониторинг по той же метрике минимум 2 недели.
- **Ретро** после релиза — короткое, с одним принятым улучшением.

3. Как это ложится на двухнедельный спринт

Спринт — две недели. Это уже заложено на доске (гипотезы отбираем в спринт раз в 2 недели, вердикт по гипотезам — до конца спринта), но на границах спринта появляются четыре правила, которых в описании этапов по умолчанию не видно. Этапы не равны спринту: этапы 1–3 укладываются в один спринт, дизайн — как правило, в следующий, разработка — в третий.

Правило 1. Крупные истории: дизайн идёт на спринт раньше разработки. Если задача попадает в разработку в спринте N, макет должен быть готов **до груминга** этого спринта — у нас это уже записано как « $\geq 80\%$ историй сложности M+ имеют прототип до груминга». Значит дизайн живёт в спринте N–1. Без этого правила получается «сегодня спека, завтра макет, послезавтра разработка», и разработка стоит день-два, ожидая макет. Цена решения: от подтверждённой проблемы до кода проходит **минимум 4 недели**.

Правило 2. Мелкие истории: дизайн внутри спринта разработки. Для S и точечных правок ждать спринт нельзя: там работа дизайнера — два часа, а не два дня. Крупные и мелкие в одном правиле смешивать нельзя: если заставить весь поток идти по правилу 1, процесс начнут обходить — и будут правы.

Правило 3. Непроверенная гипотеза не переезжает в следующий спринт автоматически. Доска требует вердикт до конца спринта, но не говорит, что делать, если вердикта нет. Правило: гипотеза не продлевается молча — её либо заново защищают и берут, либо закрывают. Иначе в спринте висят три старые гипотезы плюс новые, и ритм «1–3 гипотезы в спринт» превращается в фикцию. По объёму работы реально влезает **1–2 гипотезы, а не 3**: 5–7 интервью и выгрузка данных — это половина спринта.

Правило 4. Гейт «готово к продю» и релизы не привязаны к концу спринта. Релизы идут на клиентские стенды и в пилоты, а не по календарю спринта. Гейт — это событие «перед выкаткой», а не «в пятницу чётного спринта»; иначе он быстро станет формальностью.

Как это выглядит по спринтам:

Спринт	Что происходит
N–1	Проблема → гипотеза → проверка → Story и схема. Дизайнер читает Story и говорит, чего не хватает
N	Дизайн: 1–2 дня на макет, ревью по гайдам, саммари. В конце спринта — груминг для следующего
N+1	Разработка, промежуточная приёмка, ближе к концу — гейт «готово к продю» и выкатка
N+2	Пост-мониторинг (2 недели) и ретро

Метрики считаются за спринт: доля задач **спринта**, прошедших оба гейта; переделки — по задачам, переданным в разработку в этом спринте.

4. Живой пример: одна фича целиком

Пример собирательный, цифры условные — он нужен, чтобы показать, как этапы выглядят в жизни. По спринтам он идёт так: проблема, гипотеза и Story — спринт N–1; дизайн — спринт N, груминг в его конце; разработка и выкатка — спринт N+1.

Понедельник, утро. Поддержка приносит: за месяц 12 обращений от 4 клиентов — «заказ уехал не в тот цех, повара не поняли». Продакт нажимает одну кнопку, Hermes собирает бриф по Jira, Confluence и Pyrus: 12 обращений, 9 из них — с касс самообслуживания, похожая задача уже поднималась и была закрыта как «не воспроизводится».

Понедельник, день. Продакт формулирует проблему словами заказчика и решает: болит у четырёх клиентов, повторяется, источник есть → **подтверждена**. (Если бы жалоба была одна и разовая — отложили бы. Если бы выяснилось, что такая настройка уже есть, но ей не пользуются — сразу задача, без ресёрча.)

Вторник. Гипотеза одной страницей: «Если на карточке заказа явно показать цех, доля ошибочных переводов упадёт вдвое». Метрика — доля заказов, переведённых не в тот цех; способ проверки — данные + 5 интервью с поварами; срок — 2 недели. Приоритет по RICE — выше, чем у двух других гипотез спринта.

Среда–четверг. Проверка. Системный аналитик вытаскивает цифры: ошибочных переводов 1,8% от заказов, но в 80% случаев это одни и те же цеха. Продакт (не дизайнер) делает черновой прототип за полдня и показывает пяти поварам: четверо говорят «мы вообще не смотрим на номер цеха, он слишком мелкий». **Вердикт: проблема подтверждена, но решение другое** — дело не в отсутствии информации, а в её незаметности. Это самый дорогой момент всего примера: переделка нашлась до того, как дизайнер начал рисовать.

Пятница. Story по шаблону: источник — 12 обращений в Pyrus; бизнес-контекст — 4 клиента, кассы самообслуживания; бизнес-эффект — меньше возвратов и разборок в смене. Схему as is/to be системный аналитик собирает в Mermaid и прикладывает к задаче.

Вторник следующей недели. Дизайн. Задача на дизайн сгенерирована из Story — дизайнер открывает её и сразу видит проблему, контекст, метрику и ссылку на макет текущего экрана. Работа: два дня, по гайду компонентов (рамка карточки, размеры, состояния) и гайду текстов; тексты — черновик ИИ, дизайнер вычитывает; ревью по гайдам ДС, одна итерация доработки. В конце — саммари: «иконка цеха мигает, если заказ приходит позже трёх секунд после печати — из макета это неочевидно; после перевода показывается пустой экран, нужен текст; граничный случай — два заказа на один цех одновременно».

Середина недели. Груминг: разработка читает Story, макет и саммари. Один вопрос по поведению при параллельных заказах закрывается поиском по спеке, ответ уточняется в самой спеке — в переписке не остаётся. В середине разработки — промежуточная приёмка на тестовом стенде.

Через две недели. Гейт «готово к прод»: чек-лист, метрика есть, события собираются, стоп-условия описаны, документация передана. Запуск на трёх пилотных клиентах. Пост-мониторинг две недели: доля ошибочных переводов 1,8% → 0,7%. Ретро на 20 минут, одно принятое улучшение: в шаблон Story добавить обязательное поле «как проверим эффект».

Что в этом примере сэкономлено: дизайнер не рисовал три варианта наугад; задача на дизайн не описывалась второй раз руками; вопросы разработки не шли в личку дизайнеру; ревью не ушло в четвёртый круг; метрика измерена один раз и по делу.

5. Куда переехал каждый пункт из Miro

Этап 1 — «Понять, что болит»: правила выхода из шага (448) — правило этапа; выгрузка сигналов через дашборд (336), бот и единая форма заявки (336), база доказательств (252), ИИ-фильтр входящих идей (150) — инструменты сбора сигналов; автоматизация данных из хранилищ (120) — отдельная программа, упирается в доступы; **Item 1** — удалить.

Этап 2 — «Решить, что проверяем»: шаблон гипотезы (343), правило «без плана проверки не берём» (336) — правила; формулировка и валидация гипотез с ИИ (294), проверка гипотезы (252) — шаги; матрица «тип → способ проверки» (294) — справочник; критерии RICE и приоритизация (252 + 252) — слить в один пункт «договорились о критериях и применяем»; **Item 3** — удалить.

Этап 3 — «Описать до того, как рисовать»: обязательные поля Story/Epic (343), схема для комплексных (336), последовательность «конкуренты → схема → прототип → описание» (294), критерии точкам ветвления (180) — правила; описание Story/Epic с ИИ (336) — шаг; текстовый сценарий рядом с макетами (294) — правило дизайн-этапа; «всегда делаем прототип» (210) — сузить до сложных историй (в самой оценке сказано, что totally это лишняя работа); бриф с нейросетями (210) — инструмент; единый инструмент черновика прототипа (150) — инструмент; **Item 4** — удалить.

Этап 4 — «Дизайн»: работа этапа — задача на дизайн из Story (252), ревью по гайдам (294), лимит 3 итерации (252), саммари от дизайнера (336); база знаний — гайд по компонентам ДС (294), гайд по UX-паттернам (294), гайд по текстам (280); инструменты — скилл проверки макета (294), генерация контента для макетов (216), MCP-проверка макета (210, без доступов не стартует), перенос «макет ↔ код» (150), сводка дизайнеру по комментариям (180), MCP «Паша» (120); внедрение — пилот инструментов дизайн-этапа (336), доработка по итогам пилота (216), раскатка на продуктовый офис (210). Отдельно: бриф по Jira/Confluence/Pyrus (392) стоит в Шаге 4, а по смыслу это вход этапа 1 — перенести.

Этап 5 — «Разработка, запуск, наблюдение»: груминг (336), ответы по спеке силами ИИ (294), промежуточная приёмка (294), ретро (294) — шаги; гейт «готово к прод» (392), метрика и карта событий до пилота (336), стоп-условия (252), решение о запуске с финишем наблюдения (252), пост-мониторинг (343), ответственный за пилот и состав участников (336) — правила и точки решения; дашборд (216), дыры воронки (252), SRM-проверка (252), алерты (180), ИИ-сводка результатов пилота (150) — инструменты наблюдения.

Спецификация (бывший Шаг 5): девять пунктов — as is (343), бизнес-постановка (336), approved-версия (343), критерии приёмки по НФТ (343), границы системы и НФТ (216), варианты реализации (216), предварительный UI (216), сценарии (180), передача в разработку (294) — это **разделы одного шаблона спеки**, а не девять задач; «предварительный UI» дублирует прототип из этапа 4 — оставить в одном месте. Инструменты — скилл аналитиков для спек (294), ИИ-черновик спеки (175), требования к Spec Assistant (252), MVP процесса (192) — отдельная программа.

Внедрение процесса, а не продукта (бывший Шаг 6 и строки без шага): куратор процесса (392), раскатать на пилотных командах (224), эталонный процесс (192), AS IS текущего процесса (168), раскатать на все команды (144), ввести как обязательный (120), принять ОС от пилотов и обновить регламент (294), план бизнесового тестирования (252). Это отдельный план раскатки.

Периферия, к процессу разработки не относится: инструкция с кейсами для техписов (180), демо-кейсы и обучающий курс для клиентов (180), пул пилот-партнёров (150), единый канал коммуникации с пилотами (252), структурированный опрос по чекпоинтам (252). Свои ответственные и свой график.

Удалить или слить: Item 1, Item 3, Item 4 и пустая строка; дубль «ответы на вопросы разработки и QA» (294 + 294) — оставить один; критерии RICE (252 + 252) — один пункт.

6. Метрики

Три метрики процесса — их достаточно, чтобы понимать, работает ли он:

1. Доля задач, прошедших оба гейта. Цель $\geq 80\%$, не 100%.
2. Число переделок после передачи в разработку — снижение на 30% от базовой недели.
3. Доля Story и спек, собранных по шаблону — $\geq 80\%$.

Плюс одна метрика продукта на инициативу, зафиксированная до запуска и измеренная после — по той же самой формуле, что была в гипотезе.

7. Чем этот вариант отличается от того, что есть на доске

- **Дизайн перестаёт быть «шагом между»** — у него есть вход (задача из Story), выход (макет + саммари), норматив по времени и лимит итераций. Сейчас у дизайн-ревью и саммари нет даже строки с владельцем, хотя это самая регулярная работа дизайнера в процессе.
- **Правила отделены от шагов.** «Гипотеза без плана проверки не берётся в спринт» — это не задача на спринт, а договорённость, которая живёт в шаблоне задачи в Jira.
- **Спека собирается, а не пишется девять раз.** Один шаблон с обязательными разделами и чек-лист полноты вместо девяти строк плана.
- **Внедрение процесса — отдельный план.** Раскатка, эталонный процесс, AS IS, обучение, обратная связь пилотов: 12 пунктов, которые нужно вести своим графиком, а не мешать с продуктовыми задачами.
- **Инструменты и ИИ — отдельная программа со своими приоритетами.** MCP, скиллы, дашборды, алерты упираются в доступы к хранилищам и логам; пока доступов нет, они не «долгие», а не начатые.

Что дальше

1. Согласовать пять этапов, два гейта и правила на границах спринта (раздел 3) — это не требует ни одной новой системы, только договорённость.
2. Собрать два документа, на которых держится всё остальное: **шаблон гипотезы** и **шаблон Story** с обязательными полями. Один раз — образцы заполнения в приложении.
3. Сократить метрики до трёх и смотреть только их — формулы и место замера в приложении.
4. Перенести правила из текста в Jira: первое — условие перехода Story в статус «Ready» без обязательных полей (пример настройки в приложении).
5. Пройти по этапам и переписать формулировки так, чтобы дизайнер, продакт, аналитик и тимлид прочитали их одинаково — начинать логично с этапов 3 и 4, там сейчас больше всего переделок.
6. Отдельным планом: раскатка процесса и программа ИИ-инструментов.

Приложение. Примеры заполненных шаблонов

Все примеры — на той же фиче, что живой пример в разделе 4: заказы уезжают не в тот цех. Формы настоящие, цифры условные; там, где нужны наши данные, стоит пометка «заполнить на первом замере» — выдумывать их нет смысла.

Пример 1. Гипотеза

Поле	Что пишем
Проблема	Заказы уезжают не в тот цех: 12 обращений от 4 клиентов за месяц, источник — поддержка, Purgus
Метрика	Доля заказов, переведённых не в тот цех
Как проверяем	Выгрузка из системы + 5 интервью с поварами

Поле	Что пишем
Критерий успеха	Снижение вдвое: 1,8% → 0,9%
Срок	2 недели, вердикт до конца спринта
Тип проверки	Понятность (по матрице «тип гипотезы → способ проверки»)
Владелец	Продакт

Чего в шаблоне быть не должно: поля «предполагаемое решение» (оно фиксирует ответ до проверки) и «обоснование приоритета» текстом — приоритет считает RICE, а не абзац.

Пример 2. Story

Поле	Что пишем
Источник обращения	12 обращений в Purgus + ссылки на 3 показательных
Бизнес-контекст	4 клиента, кассы самообслуживания, ошибка повторяется в одних и тех же цехах
Бизнес-эффект	Меньше разборок в смене и возвратов: 1,8% → 0,9% ошибочных переводов
Что делаем	На карточке заказа явно показываем цех; после перевода — не пустой экран
Критерии приёмки	Given/When/Then, ниже
Схема as is/to be	Ссылка на Mermaid-схему в задаче
Макет	Ссылка на макет в Figma
Саммари от дизайнера	Ссылка на саммари (состояния и граничные случаи)

Критерии приёмки в примере:

- **Given** заказ попадает на цех, **When** повар открывает карточку, **Then** номер цеха виден в первой строке и не мельче основного текста.
- **Given** повар перевёл последний заказ, **When** список пуст, **Then** показывается текст «Нет активных заказов», а не белый экран.
- **Given** два заказа на один цех приходят одновременно, **When** повар открывает список, **Then** оба видны и не склеиваются в одну строку.

Пример 3. Правило, перенесённое в Jira

Пример формулировки, а не финальная настройка — формулировки стоит уточнить под ваш workflow:

- **Переход Story в статус «Ready»** недоступен, пока не заполнены «Источник обращения», «Бизнес-контекст», «Бизнес-эффект».
- **Переход задачи на дизайн в статус «В дизайн»** недоступен без ссылки на макет текущего состояния и на гайд по компонентам.
- **Чек-лист гейта «Готово к прод»:** метрика и карта событий зафиксированы, стоп-условия описаны, промежуточная приёмка пройдена, документация передана, расхождения с макетом разобраны.

Пример 4. Три метрики: формула и место замера

Метрика	Формула	Где считаем	База
Доля задач спринта с пройденными гейтами	задачи со статусом «Гейт пройден» ÷ все задачи спринта	фильтр в Jira, считаем в конце спринта	заполнить на первом замере
Переделки после передачи в разработку	задачи, вернувшиеся из разработки в дизайн или спеку ÷ задачи, переданные в разработку в этом спринте	журнал правок в Jira	заполнить на первом замере
Доля Story по шаблону	Story с заполненными обязательными полями ÷ все Story спринта	Jira + выборочная проверка 5 задач	заполнить на первом замере

Продуктовая метрика считается отдельно и живёт в самой инициативе: в примере — доля ошибочных переводов, замер до запуска и через две недели после.

Первый замер нужен не для отчёта, а чтобы было с чем сравнивать: если базу не зафиксировать в первом же спринте, через квартал будет не с чем сравнивать результат, и метрики превратятся в разговор о ощущениях.